Architectural Extensions

Making use of specialized components

Contents

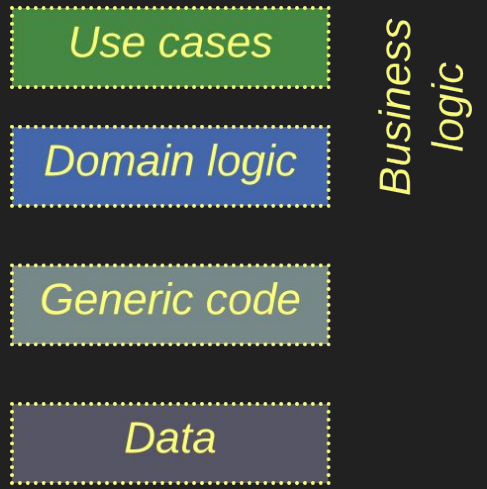
The extension layers take care of various aspects:

- *Middleware* – communication and deployment.
- *Shared Repository* – persistence and synchronization.
- *Proxy* – protocols, routing, and security.
- *Orchestrator* – integration and use cases.
- *Combined Component* – multiple aspects.

Schema

In most cases we'll be adding layers to a system of subdomain *Services*.

However, the same techniques tend to work with *Shards* or even *Layers*.

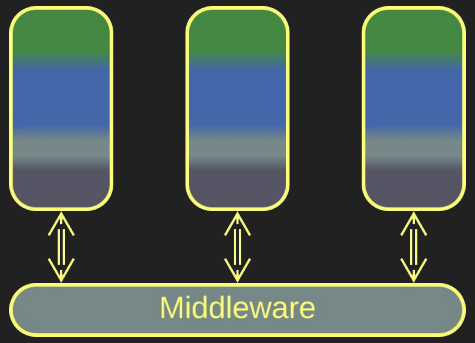


The components are color-coded

Middleware

A communication layer

Middleware – overview



Middleware

A *Middleware* is a system-wide layer which provides other system components with means of communication.

A *Middleware* may function as:

- A *Message Broker* – it lets its users message or call each other in a uniform manner.
- A *Deployment Manager* – it supervises scaling and error recovery of the services it hosts.

Middleware helps communication and scaling in complex systems.

But it may be overly generic for high-load or low latency cases.

Middleware – trade-offs

Encapsulates connectivity concerns

Takes care of scaling of components

Is available off the shelf

May increase latency

May not support specialized transports

May become a single point of failure

Middleware – addressing

A *Middleware* works in one of the following addressing modes:

- *Channels* (one to one) – two components establish a communication channel. Anything pushed to one end of the channel pops out of its other end.
- *Actors* (many to one) – every component has a public mailbox. Any component that knows its name or address can post there a message.
- *Publish / subscribe* (one to many) – a component published updates to a topic. Other components that subscribe to the topic are notified of the updates.
- *Gossip* (many to many) – every replica of a component periodically synchronizes its state with other random replica.

Middleware – flow

A *Middleware* can be used in the following ways:

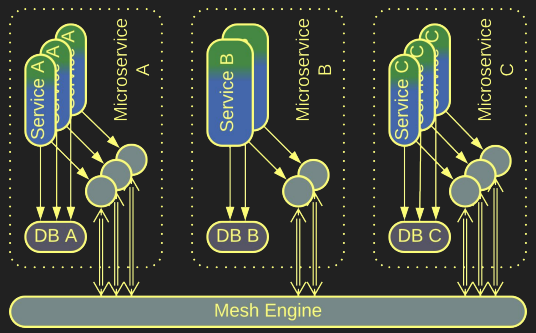
- *Notifications* – a component sends a message that indicates that an event has happened and continues its tasks. It is up to other components to subscribe to the message and continue the use case (*choreography*).
- *Request / confirm* – a component sends a request to another component (*orchestration*) and proceeds to its other tasks (*Proactor*). As soon as it receives the confirmation for its original request, it returns to its original task.
- *Remote procedure call* (*RPC*) – a component sends a request (*orchestration*) and blocks (*Reactor*) on the *Middleware* till it receives the corresponding confirmation. Then in continues its task.

Middleware – delivery guarantee

As networks and components tend to fail, messages may be lost or duplicated:

- *Exactly once* – the slowest mode of communication, often implemented with distributed transactions. Fits financial applications.
- *At least once* – messages are re-delivered on failure, risking a duplication. Therefore message processing should be *idempotent*.
- *At most once* – a message is lost on any failure. This is acceptable when the data that comes in messages is aggregated, as with weather sensors.
- *At will* (no guarantee) – raw UDP traffic that allows for message loss, duplication, and reordering. It is fast, simple, and stupid, but now the application itself must ensure message order and validity.

Middleware – composites



Service Mesh

Service Mesh is a *Mesh*-based *Middleware* with a *Proxy* per instance of client service.

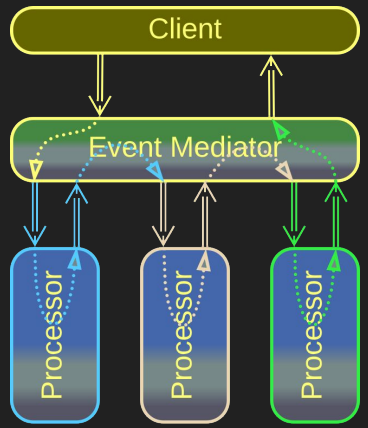
The *Proxies* contain shared libraries and interface to the *Mesh* and are co-located (*Sidecar*) with the clients.

Service Mesh is elastic – it spawns new instances of services under heavy load and deletes them when idle.

Service Mesh brings in elasticity and shared libraries.

Its implementation is complex, and you will be locked to its vendor.

Middleware – composites



Event Mediator

Event Mediator is *Middleware* + *Orchestrator*.

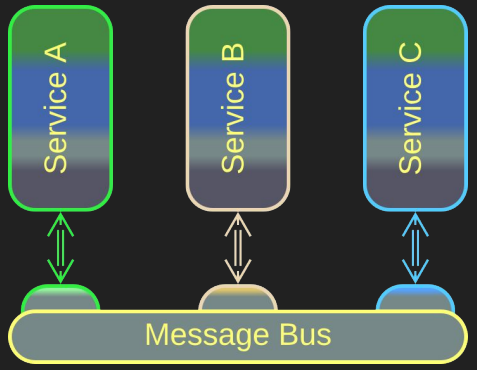
As an *Orchestrator*, it decides which message goes to which service, thereby implementing use cases.

As a *Middleware*, it provides communication, scaling, and fault recovery.

Event Mediator is a framework for *Event-Driven Architecture*.

It may grow too complex if there are many use cases.

Middleware – composites



Message Bus

Message Bus is a *Middleware* with an *Adapter* per client component.

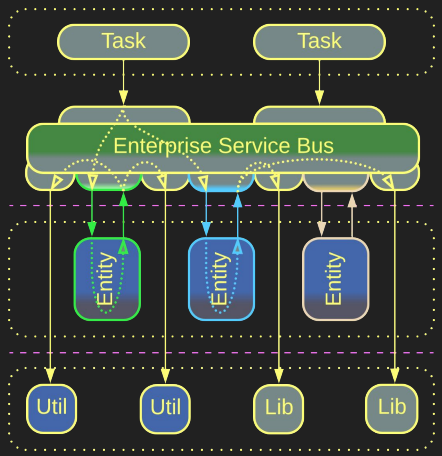
The *Adapters* translate between the components' protocols and the protocol used inside the *Message Bus*.

That enables the integration of legacy subsystems without making them communicate in a uniform manner.

Message Bus allows for integrating legacy services.

However, protocol translation slows down the communication.

Middleware – composites



Enterprise Service Bus

Enterprise Service Bus (*ESB*) is *Event Mediator* + *Message Bus*.

It orchestrates the system by knowing the destination for every kind of message, just like an *Event Mediator*.

It translates between protocols like a *Message Bus*.

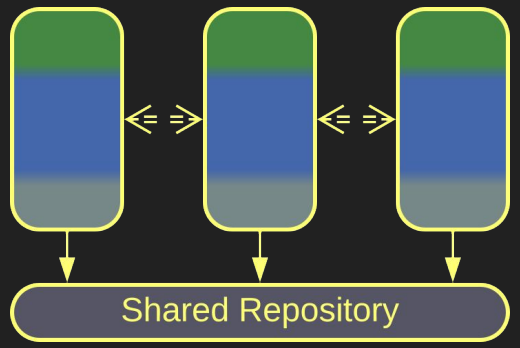
*ESB*s integrate legacy subsystems in enterprise networks.

They tend to become very complicated, undermining development.

Shared Repository

A data layer

Shared Repository – overview



Shared Repository

A Shared Repository is a data layer shared by other system components. Its functions include:

- *Persistence* – storing the data.
- *Consistency* – providing atomic transactions.
- *Notifications* – letting a component know when data changes.

Shared Repository enables data-centric or simple projects.

It may not be flexible enough for high-load or evolving domains.

Shared Repository – trade-offs

Supports coupled data

Grants data consistency

Helps saving on hardware and administration

Allows for a quick start of a project

Couples its clients to a shared schema

Has a limited scalability

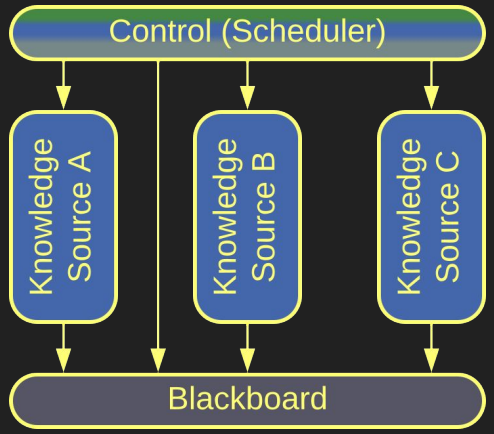
May not fit the needs of its clients equally well

May be a single point of failure

Shared Repository – variants

Pattern	Usage example
Shared memory	Instant communication in HFT
Shared file system	ETL pipelines
Shared database, integration database	Data-centric domains (seat reservation), quick start of service-based projects

Shared Repository – examples



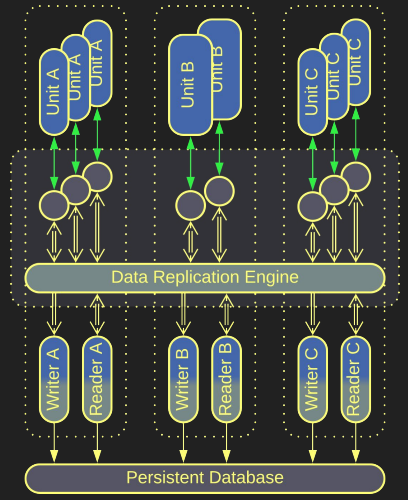
Blackboard

Blackboard was used for non-deterministic calculations.

An individual algorithm (*knowledge source*) reads whatever inputs it needs from the blackboard and adds its output to the blackboard.

The scheduler (*control*) chooses the best algorithm to run next based on the data currently available on the blackboard.

Shared Repository – examples



Data Grid

Data Grid is a virtual shared dictionary that runs in a *Mesh*. It is the foundation of *Space-Based Architecture*.

Every instance of every service in the system has a *replica* of the entire system's dataset. When a service ()writes to its *replica*, the *Data Replication Engine* copies the changes to every other system component.

The changes are also persisted to a database.

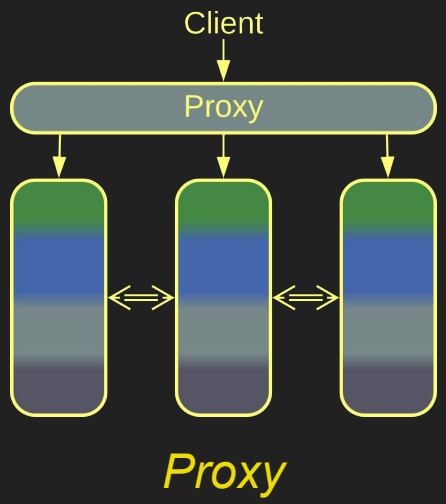
Data Grid is extremely scalable and elastic...

…but frequent writes may result in conflicts.

Proxy

Indirection between a system and its clients

Proxy – overview



Proxy

A *Proxy* stands between a system and its clients. Its aspects include:

- *Routing* – connecting a client to a component best suited to serve its needs.
- *Offloading* – implementing shared generic aspects such as protocols, logging, authentication, etc.

Proxies help with system security, scaling and protocol support…

…at the cost of slightly increased latency.

Proxy – trade-offs

Implements cross-cutting concerns

Decouples the system from its clients

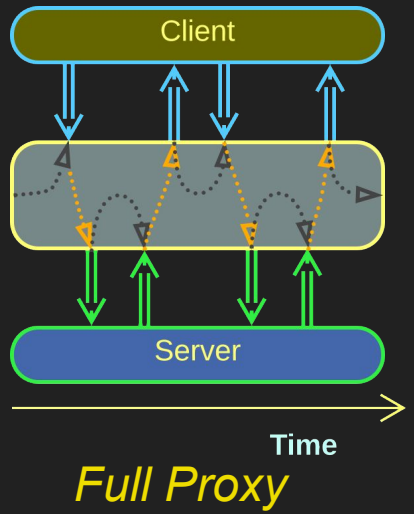
Acts as a secure perimeter

Is available off the shelf

Most kinds of *Proxies* degrade latency

May be a single point of failure

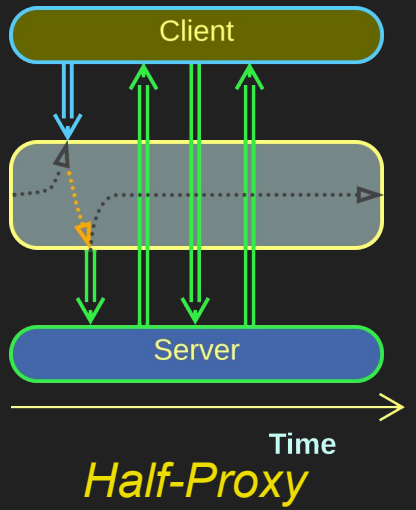
Proxy – transparency



Full Proxy

Proxies vary in their level of isolation:

- A *Full Proxy* intercepts every request between the client and the system.
- A *Half-Proxy* helps the client to establish a direct connection to a system component.

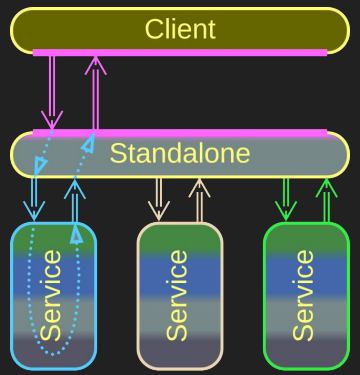


Half-Proxy

Full Proxies provide more security and control…

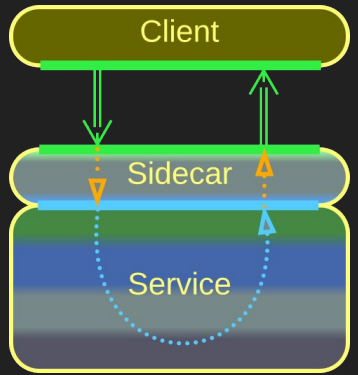
…at the cost of performance.

Proxy – placement



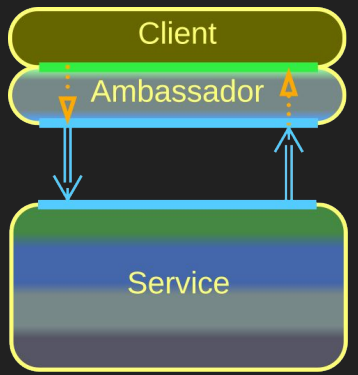
Standalone

Separate deployment



Sidecar

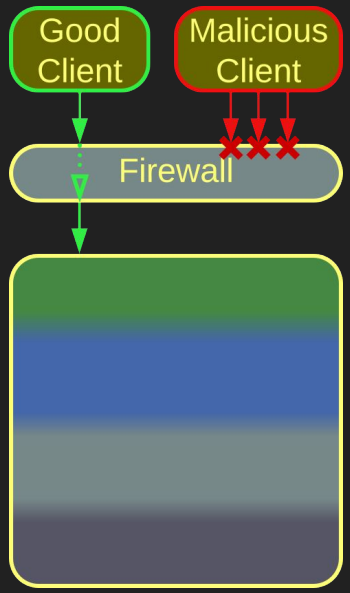
Deployed on the system side



Ambassador

Deployed on the client side

Proxy – variants



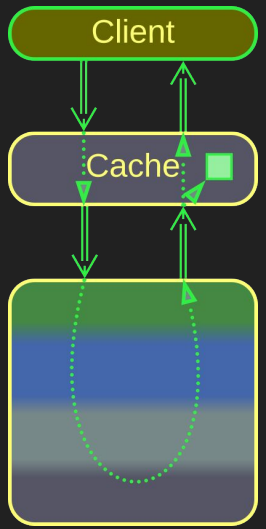
Firewall,
Rate Limiter

A *Firewall* protects the system from being overloaded by client's requests.

It should also intercept any attempts to establish a malicious connection.

A *Rate Limiter* enforces a limit on the frequency of every client's requests.

Proxy – variants



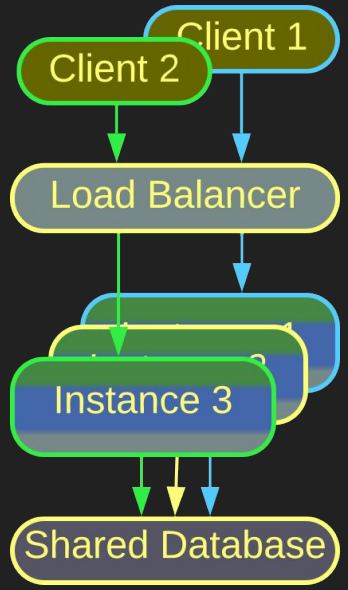
Response Cache
– not matched

A *Response Cache* remembers what the system answers to clients' requests.

If it sees a request identical to any one it has seen before, it returns the answer from its memory without consulting the underlying system.

Response Cache
– matched

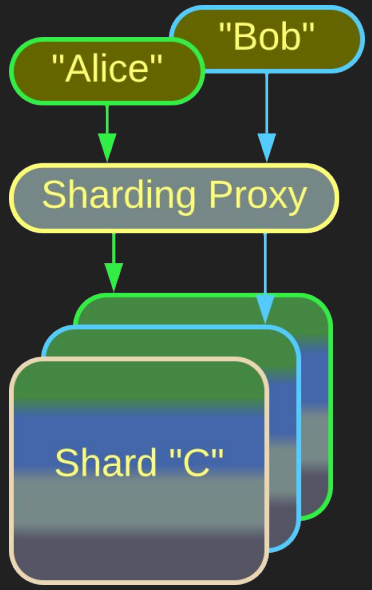
Proxy – variants



Load Balancer

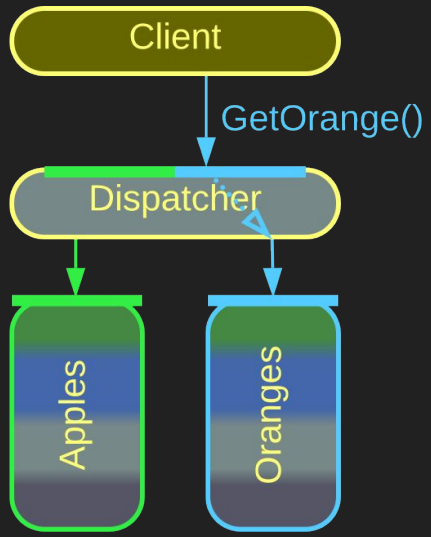
A *Load Balancer* uniformly distributes client requests among the underlying instances of a service to make sure that no instance is overloaded.

A *Sharding Proxy* connects each client to a shard that contains the client's data.



Sharding Proxy

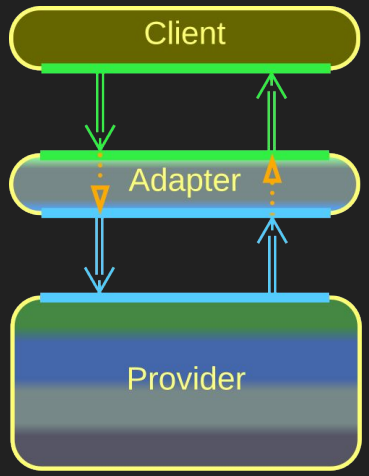
Proxy – variants



Reverse Proxy, Dispatcher

A *Reverse Proxy* forwards a client's request to the service which knows how to handle it.

Proxy – variants



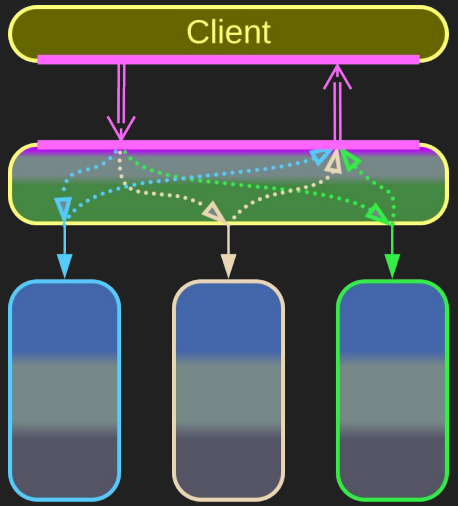
Adapter,
Abstraction Layer,
Anticorruption Layer,
Open Host Service

An *Adapter* translates between interfaces on the client's and on the system's sides.

An *Abstraction Layer* is an *Adapter* used to protect the client from changes in the interface or contract of the underlying system:

- It is called *Anticorruption Layer* when implemented by the client.
- It is called *Open Host Service* when implemented by the system's team.

Proxy – composites



API Gateway

API Gateway is an *orchestrating Proxy*:

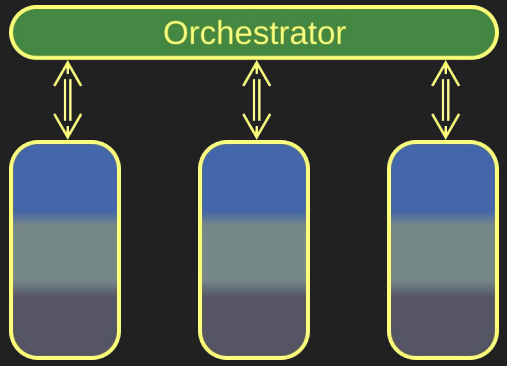
It deals with protocol translation as an *Adapter* (*Proxy*).

It splits a single client request into multiple requests to the underlying services as an *API Composer* (*Orchestrator*).

Orchestrator

An integration layer

Orchestrator – overview



Orchestrator, Application Layer

An *Orchestrator* integrates lower-level components. It acts as:

- A *Mediator* by propagating state changes among system components.
- A *Facade* by implementing system-wide use cases.

Orchestrator helps projects with many complex use cases…

…at the cost of increased latency and operational complexity.

Orchestrator – trade-offs

Encapsulates integration concerns

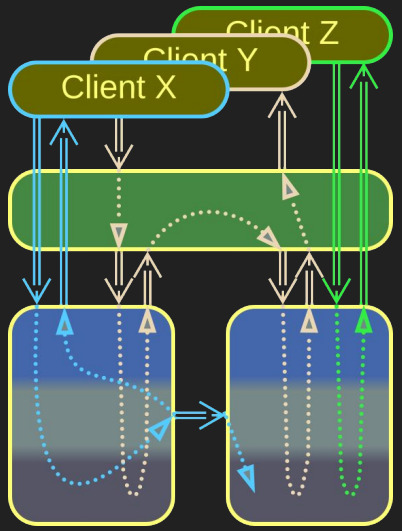
Simplifies making changes to use cases

Protects the teams behind lower-level components from the system's clients

May degrade the latency and fault tolerance of the system

May grow too large for comfortable development

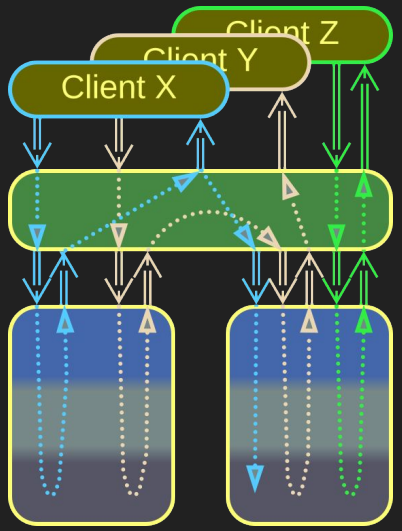
Orchestrator – transparency



Open
Orchestrator

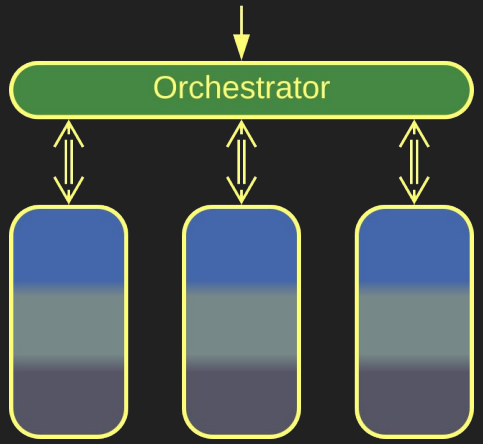
Being a layer, an *Orchestrator* can allow or block its clients' access to the underlying components:

- An *Open Orchestrator* implements complex use cases but passes simple client requests to the services below it.
- A *Closed Orchestrator* intercepts all client requests, even those that don't involve multiple services.



Closed
Orchestrator

Orchestrator – internals



Monolithic Orchestrator

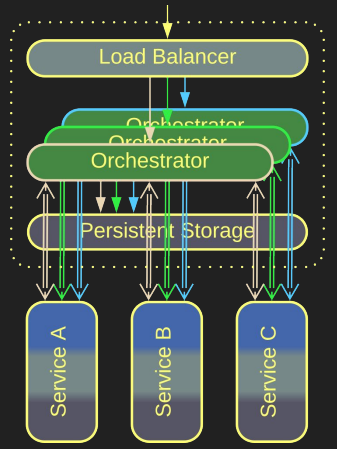
In the simplest case an *Orchestrator* is monolithic (or layered if it persists its state to a database for fault recovery).

The single-component structure is good for smaller projects with no stringent requirements.

The monolithic structure is simple and its state is self-consistent…

…but it has limited performance and may become too large.

Orchestrator – internals



Scaled Orchestrator

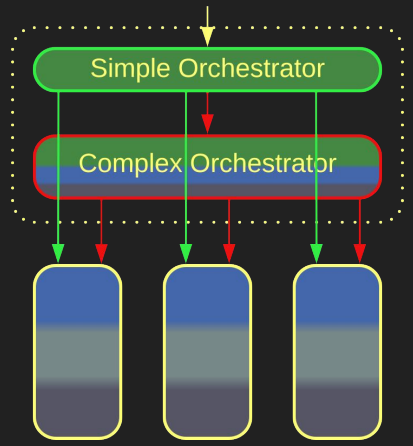
High availability or high load require multiple instances of an *Orchestrator*.

The instances rely on a *Shared Repository* or *replication* to synchronize their states.

Setting up multiple instances increases throughput and availability.

However, the instances must take care to avoid race conditions.

Orchestrator – internals



Layered Orchestrator

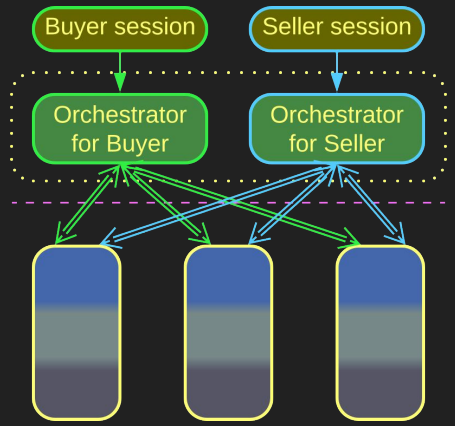
It is possible to use several *Orchestrators* of different complexity.

All user requests get to a simple *Orchestrator*. If it cannot handle a request, it forwards the latter to a more complex implementation.

A *Layered Orchestrator* reserves complex code for hard use cases.

But it requires the team to learn multiple technologies.

Orchestrator – internals



Backends for Frontends

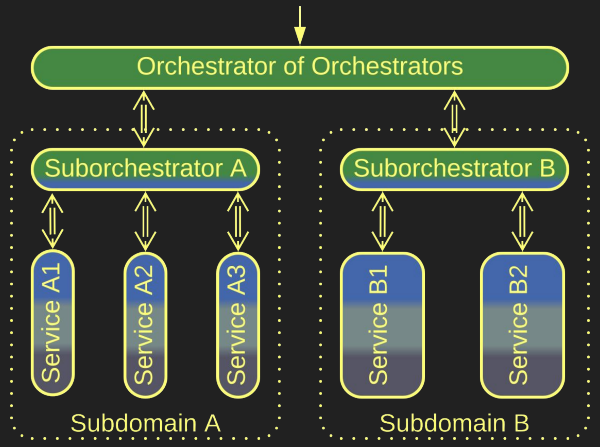
The orchestration layer may dedicate one service to each kind of client, becoming *Backends for Frontends* (*BFF*).

For example, an online store may implement buyer and seller logic with separate *Orchestrators*.

BFF is good when clients differ in their use cases.

On the downside, it is hard to share code between use cases.

Orchestrator – internals



Top-Down Hierarchy

Top-Down Hierarchy has multiple layers of *Orchestrators*.

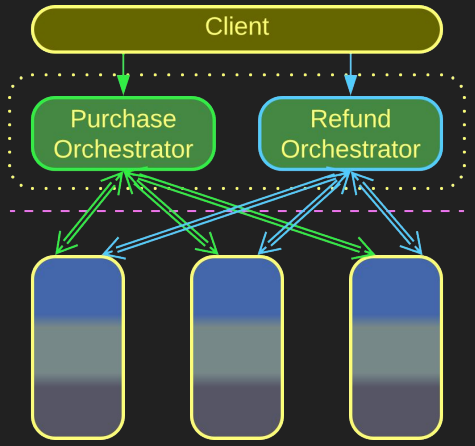
The top-level component operates coarse-grained actions.

Each layer translates a call to its API into several calls to the APIs of the layer below it.

Hierarchy distributes complexity among several layers.

However, only few domains are hierarchical.

Orchestrator – internals



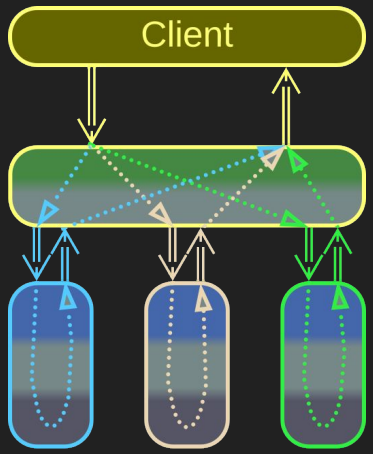
SOA-style orchestration

The *SOA*-style orchestration relies on many small *Orchestrators*, each implementing a few closely related scenarios.

Each *Orchestrator* stays small and humble.

But there are too many of them.

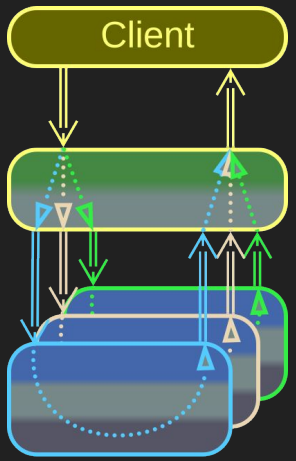
Orchestrator – variants



API Composer

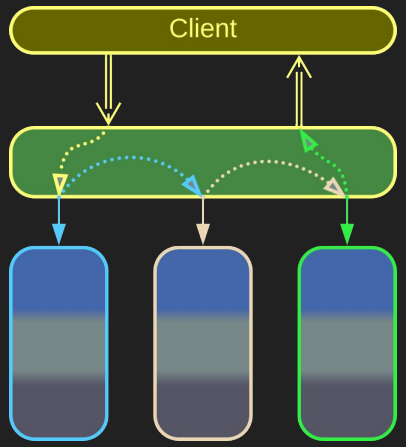
An *API Composer* parses a client's request, forwards its parts to the underlying services, and merges the results into a single response to its client.

With *Scatter/Gather* or *MapReduce* a copy of the incoming request is forwarded to each shard, and the results are aggregated and sent to the client as a single response.



Scatter/Gather, MapReduce

Orchestrator – variants

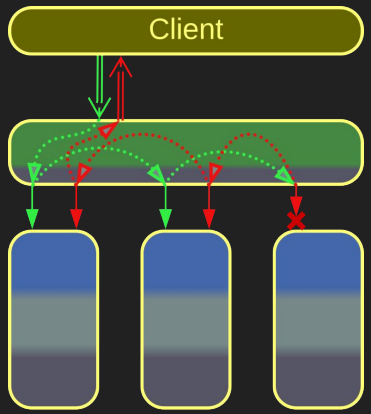


Process Manager

Unlike *API Composer*, which runs actions in parallel, *Process Manager* executes a use case as a sequence of steps.

It can have complex logic with branching and error handling.

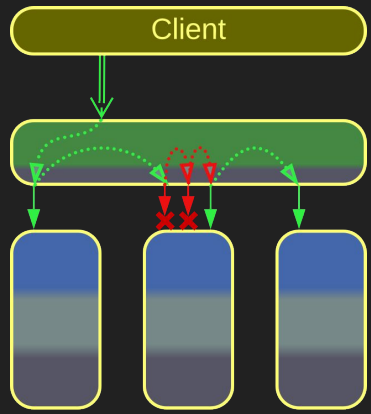
Orchestrator – variants



Atomically Consistent Saga

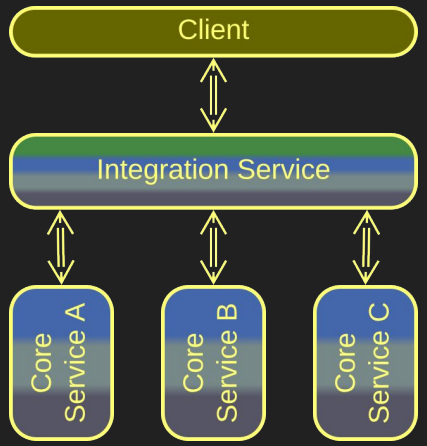
A *Saga* is a scenario that updates states of multiple components:

- An *Atomically Consistent Saga* rolls back changes on any error.
- An *Eventually Consistent Saga* retries applying the changes till it succeeds.



Eventually Consistent Saga

Orchestrator – variants



Integration Service, Application Service

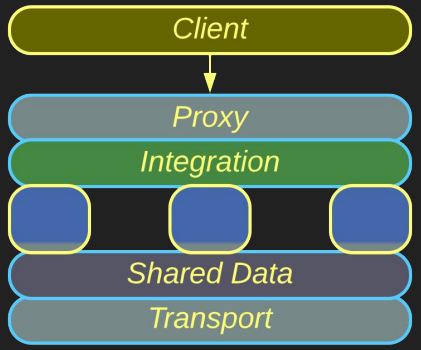
An *Integration Service* is a full-featured service which uses other services in its operation.

It tends to implement the application layer (use case logic) in DDD terminology.

Combined Component

A multifunctional layer

Combined Component – overview



Combined Component

A *Combined Component* implements the functionality of several different system-wide layers.

Combined Components expedite programming in familiar domains.

It is very hard to replace if you outgrow it.

Combined Component – trade-offs

Covers many of your needs off the shelf

Has better performance and simpler setup than a set of stand-alone layers

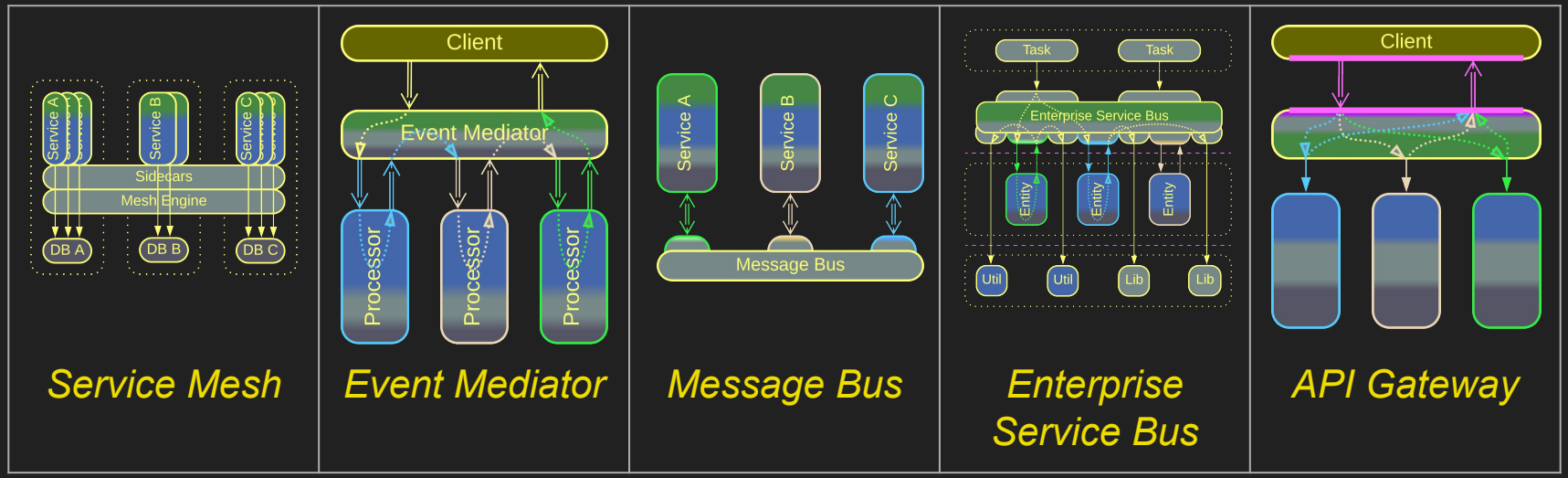
Locks you to its vendor

Requires learning a proprietary technology

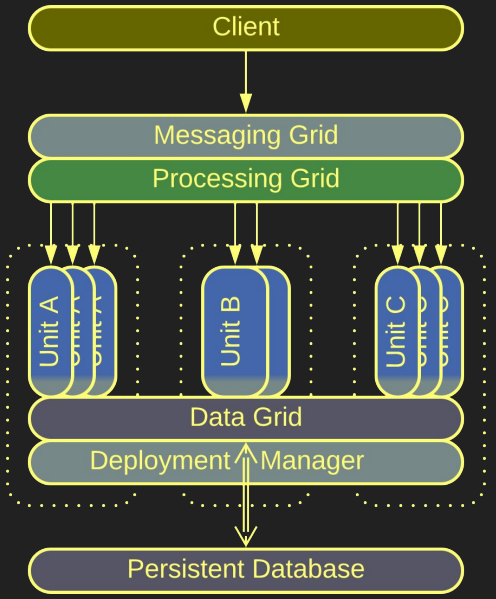
May be too simple, too complex, or just misaligned for your needs

Combined Component – examples

We have already covered several Combined Components:



Combined Component – examples

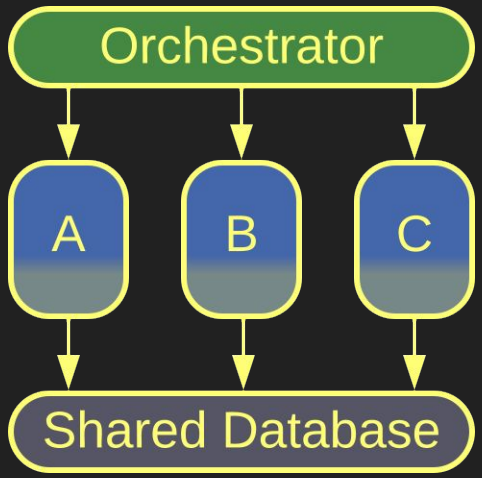


Space-Based Architecture

Space-Based Architecture (*SBA*) provides a complex framework that contains:

- *Messaging Grid* – a *Proxy* that processes client requests.
- A *Processing Grid* – an *Orchestrator* for complex workflows.
- A *Data Grid* – a distributed *Shared Repository*.
- A *Deployment Manager* – a *Middleware*.

Combined Component – examples



Cell

A *Cell* is an intermediate step in transition from *Layers* to *Services*.

The domain logic layer is the easiest to subdivide into services, while other components remain as layers.

Though not being genuine *Combined Components*, *Cells* show a similar structure.

Conclusion

Moving functionality out of services

Each of the extension patterns which we've reviewed takes care of some functionality which would otherwise need to be implemented by the main services that contain the business logic.

It is the business logic that makes the core of the business and brings profit to the company. Your teams should concentrate their efforts on developing it. Everything else is expendable and should be acquired when available off the shelf.

Segregating technical or behavioral aspects not only lowers the development complexity, but may also decouple the subdomains, as we see when data-centric systems are built around *Shared Repositories* or when applications with complex use cases employ *Orchestrators*.

Links

This was an overview of part 3 of my book *Architectural Metapatterns* which is available:

– as a website metapatterns.io

– as a PDF or EPUB from leanpub.com/metapatterns

More patterns are to follow…